BASH SCRIPTING FUNDAMENTALS

Functions

Reusable building blocks and clean script structure

Merit Advisory

Bash Scripting Fundamentals Bootcamp

Beginner-friendly lecture materials

July 2026

WHAT WE WILL COVER

Agenda

1. Section 1: Function Foundations — 7 topics
2. Section 2: Function Parameters — 7 topics
3. Section 3: Scope And Local State — 7 topics
4. Section 4: Return Values And Output — 7 topics
5. Section 5: Main Pattern And Libraries — 7 topics
6. Section 6: Error Logging And Assertions — 7 topics
7. Section 7: Advanced Function Mechanics — 7 topics
8. Section 8: Function Introspection — 7 topics

9 Section 9: Manual Testing And Documentation — 7 topics

10 Section 10: Refactoring Into Functions — 7 topics

11 Section 11: Data Flow Techniques — 7 topics

12 Section 12: Checkpoint Labs — 7 topics

1 Section 1

Function Foundations

1.1 Why functions

- Functions name a reusable piece of shell logic.

Repeated action

```
say_hi() { echo "hi $1"; }
say_hi Ada
```

- They reduce repeated code in longer scripts.

Named step

```
build() { make all; }
build
```

1.2 Name parentheses braces form

- The name() { } form is common Bash function syntax.

Define

```
hello() { echo hello; }
```

- A space is required before the opening brace body.

Define and call

```
hello() { echo hello; }
hello
```

1.3 Function keyword form

- Bash also accepts the function keyword.

Keyword

```
function hello { echo hello; }
```

- It is less portable but common in Bash-only scripts.

Keyword call

```
function build { echo build; }
build
```

1.4 Calling functions

- Call a function by writing its name like a command.

Simple call

```
greet() { echo hi; }
greet
```

- Arguments follow the name separated by spaces.

Call with arg

```
greet() { echo "hi $1"; }
greet Sam
```

1.5 Declare before use

- Bash executes scripts from top to bottom.

Before main

```
helper() { echo help; }
helper
```

- A function definition must be seen before it is called.

Main last

```
main() { helper; }
helper() { echo h; }
```

1.6 Function call status

- A function returns the status of its last command by default.

Status from grep

```
has_root() { grep -q root /etc/passwd; }
has_root && echo yes
```

- That status can drive if, &&, and ||.

Status from test

```
exists() { [[ -e $1 ]]; }
exists file || echo missing
```

1.7 Compose small functions

- Small functions are easier to name and reuse.

Two helpers

```
clean() { rm -rf out; }
build() { mkdir out; }
```

- One function should do one clear job.

Workflow

```
run_all() { clean && build; }
run_all
```

2 Section 2

Function Parameters

2.1 First argument inside a function

- $1 inside a function is the function first argument.

Print first

```
show() { echo "$1"; }
show alpha
```

- It is separate from the script first argument.

Use path

```
size() { wc -c < "$1"; }
size file.txt
```

2.2 All arguments with dollar at

- "$@" expands to all function arguments safely.

Forward

```
run() { command "$@"; }
run echo hi
```

- Each original argument stays a separate word.

List args

```
show() { printf '%s\n' "$@"; }
show a "b c"
```

2.3 Argument count

- $# is the number of arguments passed to the function.

Require one

```
need_one() { [[ $# -eq 1 ]] || return 2; }
```

- Use it for arity checks.

Print count

```
count_args() { echo "$#"; }
count_args a b
```

2.4 Default arguments

- Parameter expansion can provide defaults.

Default env

```
deploy() { local env=${1:-dev}; echo "$env"; }
```

- Defaults make optional parameters explicit.

Default file

```
read_cfg() { local cfg=${1:-config.env}; echo "$cfg"; }
```

2.5 Shift inside functions

- shift removes the first function argument.

Drop command

```
run_cmd() { local cmd=$1; shift; "$cmd" "$@"; }
```

- It is useful after parsing an option or command.

Parse first

```
show_rest() { local first=$1; shift; printf '%s\n' "$@"; }
```

2.6 Validate function arguments

- Functions should check the inputs they require.

Require file

```
read_file() { [[ -r $1 ]] || return 2; wc -l < "$1"; }
```

- Return a nonzero status when validation fails.

Require int

```
twice() { [[ $1 =~ ^[0-9]+$ ]] || return 2; echo $(($1*2)); }
```

2.7 Usage function

- A usage function prints the valid command form.

Usage text

```
usage() { echo "usage: app FILE"; }
```

- Keep it short enough to read after an error.

Use usage

```
[[ $# -eq 1 ]] || { usage; exit 2; }
```

3 Section 3

Scope And Local State

3.1 Local variables

- local creates a variable scoped to the current function.

Local name

```
greet() { local name=$1; echo "hi $name"; }
```

- It prevents accidental changes to global variables.

Local count

```
count() { local n=0; ((n++)); echo $n; }
```

3.2 Missing local pitfall

- Assignments without local can change global variables.

Global leak

```
tmp=old
set_tmp() { tmp=new; }
```

- This can create surprising behavior between functions.

Fixed

```
set_tmp() { local tmp=new; echo "$tmp"; }
```

3.3 Readonly function variables

- readonly prevents reassignment after a value is set.

Local readonly

```
show() { local -r path=$1; echo "$path"; }
```

- Inside functions, combine local and readonly for constants.

Readonly global

```
readonly APP_NAME=demo
```

3.4 Global variables are shared state

- Global variables are visible inside functions.

Read global

```
prefix=app
name() { echo "$prefix-$1"; }
```

- They couple helpers to outer script state.

Avoid write

```
set_env() { local env=$1; echo "$env"; }
```

3.5 Local arrays

- Arrays can be local to a function.

Local array

```
list() { local -a xs=(a b); printf '%s\n' "${xs[@]}"; }
```

- Use local -a for indexed arrays.

Build array

```
make_args() { local -a args=(--name "$1"); printf '%s\n' "${args[@]}"; }
```

3.6 Nameref intro

- local -n creates a nameref to another variable.

Set by name

```
set_value() { local -n ref=$1; ref=$2; }
set_value result ok
```

- It lets a function update a caller variable by name.

Array by name

```
first_item() { local -n arr=$1; echo "${arr[0]}"; }
```

3.7 Unset temporary names

- unset removes a variable name from the current scope.

Unset local

```
demo() { local token=$1; unset token; }
```

- Local variables disappear automatically when a function returns.

Unset global

```
cache=value
unset cache
```

4 Section 4

Return Values And Output

4.1 Return status

- return sets the function exit status.

Return success

```
ok() { return 0; }
ok && echo yes
```

- A status of 0 means success.

Return failure

```
fail() { return 1; }
fail || echo no
```

4.2 Return status range

- Shell return statuses are small integers.

Bad data return

```
get_name() { echo Ada; }
```

- Do not use return for strings or structured data.

Status only

```
is_file() { [[ -f $1 ]]; }
```

4.3 Return versus echo

- return communicates success or failure.

Predicate

```
is_prod() { [[ $1 == prod ]]; }
```

- echo or printf communicates textual output.

Producer

```
make_name() { printf 'app-%s\n' "$1"; }
```

4.4 Capture command substitution

- Command substitution captures stdout from a function.

Capture

```
now() { date +%s; }
ts=$(now)
```

- Trailing newlines are removed by the substitution.

Capture name

```
slug() { echo "app-$1"; }
name=$(slug api)
```

4.5 Stdout versus stderr

- Stdout is for data a caller may capture.

Error output

```
warn() { echo "warn: $*" >&2; }
```

- Stderr is for diagnostics and errors.

Data output

```
value() { printf '%s\n' "$1"; }
```

4.6 Quiet predicate functions

- Predicate functions often print nothing.

Has command

```
has_cmd() { command -v "$1" >/dev/null 2>&1; }
```

- Their status is the answer.

Readable

```
can_read() { [[ -r $1 ]]; }
```

4.7 Multiple values with echo

- A function can print multiple lines as output.

Two lines

```
pair() { printf '%s\n' name Ada; }
```

- The caller must parse that output deliberately.

Parse simple

```
vals=$(pair)
printf '%s\n' "$vals"
```

5 Section 5

Main Pattern And Libraries

5.1 Main function pattern

- A main function names the script entry point.

Main

```
main() { echo run; }
main "$@"
```

- It keeps top-level code short.

Main status

```
main() { return 0; }
main "$@"
```

5.2 Pass arguments to main

- Use "$@" when calling main from top level.

Forward all

```
main() { printf '%s\n' "$@"; }
main "$@"
```

- This preserves the original script arguments.

Use first

```
main() { echo "cmd=$1"; }
main "$@"
```

5.3 Sourcing a library

- source reads another shell file into the current shell.

Source lib

```
source ./lib.sh
helper
```

- Functions defined there become available to the script.

Dot form

```
. ./lib.sh
helper
```

5.4 Guard against re source

- A guard variable can prevent loading a library twice.

Library guard

```
[[ ${LIB_LOADED:-} ]] && return 0
LIB_LOADED=1
```

- Set the guard after the first successful load.

Readonly guard

```
[[ ${UTILS_LOADED:-} ]] && return 0
readonly UTILS_LOADED=1
```

5.5 Function library layout

- Group related helper functions in a small library file.

Utils file

```
log_info() { echo "info: $*"; }
```

- Keep script entry logic out of reusable libraries.

Script use

```
source ./utils.sh
log_info start
```

5.6 Helpers before main

- Place helper definitions before main calls them.

Order

```
helper() { echo h; }
main() { helper; }
```

- This matches Bash top to bottom execution.

Call last

```
main() { echo run; }
main "$@"
```

5.7 Run only when executed

- BASH_SOURCE can distinguish source from execute.

Entry guard

```
if [[ ${BASH_SOURCE[0]} == $0 ]]; then main "$@"; fi
```

- This allows a file to be both script and library.

Source safe

```
main() { echo run; }
[[ ${BASH_SOURCE[0]} == $0 ]] && main "$@"
```

6 Section 6

Error Logging And Assertions

6.1 Die helper

- A die helper prints an error and exits.

Die

```
die() { echo "error: $*" >&2; exit 1; }
```

- It centralizes fatal error formatting.

Use die

```
[[ -r $cfg ]] || die "missing config"
```

6.2 Error helper

- An error helper prints to stderr without exiting.

Error

```
error() { echo "error: $*" >&2; }
```

- It lets callers decide whether to return or continue.

Return after error

```
need_file() { [[ -f $1 ]] || { error missing; return 2; }; }
```

6.3 Info logging helper

- An info helper marks normal progress messages.

Info

```
info() { echo "info: $*" >&2; }
```

- Send logs to stderr when stdout is data.

Use info

```
info "starting build"
```

6.4 Warn logging helper

- A warn helper marks nonfatal problems.

Warn

```
warn() { echo "warn: $*" >&2; }
```

- It should not exit on its own.

Use warn

```
[[ -f optional.env ]] || warn "no optional env"
```

6.5 Assert command helper

- An assert helper checks an invariant during a script.

Assert

```
assert() { "$@" || die "assert failed: $*"; }
```

- Failed assertions should return or exit clearly.

Assert use

```
assert test -d src
```

6.6 Assert file helper

- A file assertion names a common required condition.

Require file

```
require_file() { [[ -f $1 ]] || return 2; }
```

- It keeps call sites short and readable.

Use require

```
require_file config.env || die "need config"
```

6.7 Command wrapper

- A wrapper function adds policy around another command.

Git wrapper

```
git_checked() { git "$@" || die "git failed"; }
```

- It can add logging, defaults, or validation.

Curl wrapper

```
fetch() { curl -fsS "$@"; }
```

7 Section 7

Advanced Function Mechanics

7.1 Functions and exit

- exit inside a function exits the shell or script.

Return

```
check() { [[ -e $1 ]] || return 2; }
```

- return exits only the function.

Exit fatal

```
fatal() { echo bad >&2; exit 1; }
```

7.2 Set e inside functions

- set -e can interact with function calls in surprising ways.

Explicit return

```
copy() { cp "$1" "$2" || return; }
```

- Commands tested by if or || are treated specially.

If tested

```
if risky_fn; then echo ok; fi
```

7.3 Err trap caution

- ERR traps can run when commands fail under set -e.

Err trap

```
trap 'echo failed >&2' ERR
set -E
```

- Functions make trap flow harder to reason about.

Function failure

```
work() { false; }
work
```

7.4 Recursion caution

- Bash functions can call themselves recursively.

Recursive shape

```
countdown() { (( $1 == 0 )) || countdown $(($1-1)); }
```

- Deep recursion is usually a poor fit for shell scripts.

Loop alternative

```
for ((n=3; n>=0; n--)); do echo $n; done
```

7.5 Overriding commands

- A function can have the same name as a command.

Wrapper

```
ls() { command ls --color=auto "$@"; }
```

- This changes what unqualified calls run.

Bypass

```
command ls
```

7.6 Builtin command bypass

- The command builtin skips shell function lookup.

Call original

```
grep() { command grep --color=auto "$@"; }
```

- It is useful inside wrappers that call the original command.

Find command

```
command -v bash >/dev/null
```

7.7 Unset function

- unset -f removes a function definition.

Remove function

```
tmp_fn() { echo tmp; }
unset -f tmp_fn
```

- It is useful in interactive shells and tests.

Replace wrapper

```
unset -f ls
```

8 Section 8

Function Introspection

8.1 Declare f

- declare -f prints function definitions.

Show function

```
hello() { echo hi; }
declare -f hello
```

- It is useful for debugging sourced libraries.

All functions

```
declare -f
```

8.2 Type t function

- type -t reports what kind of name Bash resolves.

Check type

```
hello() { :; }
type -t hello
```

- It prints function for function names.

Command type

```
type -t cd
```

8.3 Command V

- command -V describes how a name will be interpreted.

Describe

```
hello() { :; }
command -V hello
```

- It can show functions, builtins, aliases, and files.

Describe ls

```
command -V ls
```

8.4 Declare F

- declare -F lists function names without bodies.

List names

```
declare -F
```

- It is shorter than declare -f for inventories.

Has name

```
declare -F helper >/dev/null && echo loaded
```

8.5 Exported functions caution

- export -f can pass functions to child Bash processes.

Export function

```
helper() { echo h; }
export -f helper
```

- It is Bash specific and should be used sparingly.

Child bash

```
bash -c helper
```

8.6 Readonly functions

- readonly -f prevents a function from being redefined or unset.

Protect

```
die() { exit 1; }
readonly -f die
```

- It can protect critical helpers in controlled scripts.

Check readonly

```
readonly -f
```

8.7 Namespace prefixes

- Prefixes reduce function name collisions.

Prefixed log

```
log_info() { echo "info: $*"; }
```

- They are helpful in sourced libraries.

Prefixed path

```
path_join() { echo "$1/$2"; }
```

9 Section 9

Manual Testing And Documentation

9.1 Manual function calls

- Source a file and call functions directly while developing.

Source then call

```
source ./script.sh
my_fn test
```

- This shortens the edit and test loop.

Call failure

```
my_fn missing || echo failed
```

9.2 Interactive source testing

- An interactive shell can load your helpers with source.

Load

```
source ./lib.sh
declare -F
```

- declare -f confirms the definitions are present.

Cleanup

```
unset -f helper
```

9.3 Temporary argument tests

- Small test calls can exercise argument handling.

Space arg

```
show() { printf '<%s>\n' "$@"; }
show "a b"
```

- Include spaces in sample arguments.

Status arg

```
need_one() { [[ $# -eq 1 ]]; }
need_one a b || echo bad
```

9.4 Capture output tests

- Command substitution can check function output.

Output check

```
got=$(slug api)
[[ $got == app-api ]]
```

- Compare captured output to the expected value.

Print failure

```
[[ $got == ok ]] || echo "got $got"
```

9.5 Compare status tests

- Status tests check whether predicates succeed or fail.

Predicate pass

```
is_num() { [[ $1 =~ ^[0-9]+$ ]]; }
is_num 42 && echo pass
```

- Use if or && and || for manual checks.

Predicate fail

```
is_num x || echo fail
```

9.6 ShellCheck SC2155 note

- SC2155 warns about declaring and assigning in one command.

Separated local

```
local value
value=$(cmd)
```

- Separate declaration from assignment when status matters.

Simple local

```
local name=$1
```

9.7 Documentation comments

- A short comment can describe a function contract.

Contract

```
# Prints a slug for one name.
slug() { echo "app-$1"; }
```

- Mention inputs, output, and status when helpful.

Status comment

```
# Returns 0 when the path is readable.
can_read() { [[ -r $1 ]]; }
```

10

Section 10

Refactoring Into Functions

10.1 Extract repeated code

- Repeated command sequences are good function candidates.

Extract

```
backup() { cp "$1" "$1.bak"; }
```

- Name the extracted behavior by its purpose.

Reuse

```
backup app.conf
backup db.conf
```

10.2 Pure functions

- A pure style uses parameters and stdout without hidden state.

Pure slug

```
slug() { printf '%s\n' "${1// /-}"; }
```

- It is easier to test and reuse.

Capture pure

```
name=$(slug "api server")
```

10.3 Side effect functions

- Side effect functions change files, directories, or process state.

Make directory

```
ensure_dir() { [[ -d $1 ]] || mkdir -p "$1"; }
```

- Name them with verbs so the change is obvious.

Remove file

```
remove_file() { [[ -f $1 ]] && rm "$1"; }
```

10.4 Limit global dependencies

- Functions that rely on globals are harder to reuse.

Pass path

```
load_cfg() { source "$1"; }
```

- Pass paths and options as arguments instead.

Avoid hidden path

```
run_tool() { local bin=$1; shift; "$bin" "$@"; }
```

10.5 Small predicate helpers

- Predicate helpers wrap common tests behind clear names.

Predicate

```
is_readable_file() { [[ -f $1 && -r $1 ]]; }
```

- They should return status and usually print nothing.

Use predicate

```
if is_readable_file config; then echo ok; fi
```

10.6 Wrap pipelines

- A pipeline can be hidden behind a named function.

Count errors

```
count_errors() { grep -c ERROR "$1"; }
```

- The name documents what the pipeline produces.

Top names

```
top_names() { cut -d, -f1 "$1" | sort; }
```

10.7 Teardown helpers

- Cleanup code is a good candidate for a helper.

Cleanup

```
cleanup() { rm -rf "$tmpdir"; }
```

- A named cleanup function can be used with trap.

Trap cleanup

```
trap cleanup EXIT
```

11 Section 11

Data Flow Techniques

11.1 Printf v assignment

- printf -v assigns formatted text to a variable by name.

Assign formatted

```
printf -v name 'app-%s' "$1"
```

- It avoids command substitution for simple formatting.

Inside function

```
make_name() { printf -v result 'app-%s' "$1"; }
```

11.2 Nameref output

- A nameref can act like an output parameter.

Output variable

```
make_name() { local -n out=$1; out=app; }
make_name result
```

- The caller passes the variable name to update.

Status with output

```
find_cfg() { local -n out=$1; out=config.env; }
```

11.3 Global output discouraged

- Writing results into globals couples caller and function.

Global result

```
RESULT=
set_result() { RESULT=ok; }
```

- It can be acceptable in tiny scripts but scales poorly.

Stdout result

```
get_result() { echo ok; }
```

11.4 Arrays passed by name

- Bash arrays are often passed to functions by variable name.

Read array

```
first() { local -n a=$1; echo "${a[0]}"; }
```

- local -n lets the function read or update that array.

Append array

```
add_item() { local -n a=$1; a+=("$2"); }
```

11.5 Read loop in function

- A function can own a while read loop.

Print lines

```
print_file() { while IFS= read -r l; do echo "$l"; done < "$1"; }
```

- Pass the input path as an argument.

Count lines

```
count_file() { local n=0; while read -r _; do ((n++)); done < "$1"; echo $n; }
```

11.6 Mapfile in function

- mapfile can load function input into a local array.

Local mapfile

```
load_lines() { local -a lines; mapfile -t lines < "$1"; echo "${#lines[@]}"; }
```

- Use -t to remove trailing newlines.

Print first

```
first_line() { local -a l; mapfile -t l < "$1"; echo "${l[0]}"; }
```

11.7 Stable output format

- Function output becomes an interface for callers.

Key value

```
emit_pair() { printf '%s=%s\n' "$1" "$2"; }
```

- Changing columns or separators can break scripts.

One per line

```
emit_list() { printf '%s\n' "$@"; }
```

12 Section 12

Checkpoint Labs

12.1 Build usage lab

- Write a usage function for one script command.

Usage

```
usage() { echo "usage: app build|test"; }
```

- Call it when arguments are missing or invalid.

Usage error

```
[[ $# -gt 0 ]] || { usage; exit 2; }
```

12.2 Refactor script lab

- Find two repeated command groups in a script.

Extract build

```
build_app() { make build; }
```

- Extract each group into a named function.

Main uses

```
main() { build_app && test_app; }
```

12.3 Validator lab

- Create predicate functions for common input checks.

Integer predicate

```
is_int() { [[ $1 =~ ^-?[0-9]+$ ]]; }
```

- Keep predicate output quiet.

Path predicate

```
is_dir() { [[ -d $1 ]]; }
```

12.4 Logging library lab

- Create info, warn, and error helpers.

Info warn

```
info() { echo "info: $*" >&2; }
warn() { echo "warn: $*" >&2; }
```

- Send all log messages to stderr.

Error

```
error() { echo "error: $*" >&2; }
```

12.5 Retry function lab

- Wrap retry behavior in a function.

Retry shape

```
retry() { for i in 1 2 3; do "$@" && return 0; done; return 1; }
```

- Accept the command and its arguments with "$@".

Use retry

```
retry curl -fsS http://localhost
```

12.6 Dispatch function lab

- Use case inside main to choose a function.

Dispatch

```
case $1 in build) build ;; test) test_all ;; *) usage ;; esac
```

- Keep command names separate from implementation details.

Function command

```
build() { echo build; }
test_all() { echo test; }
```

12.7 Function checklist

- Name the function by the job it performs.

Checklist helper

```
has_file() { [[ -f $1 ]]; }
```

- Declare local variables for temporary state.

Checklist output

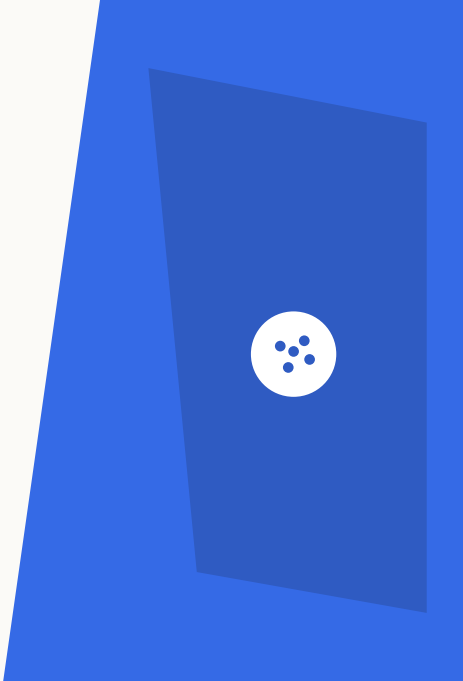
```
make_slug() { printf '%s\n' "${1// /-}"; }
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory
04 · Functions



July 2026